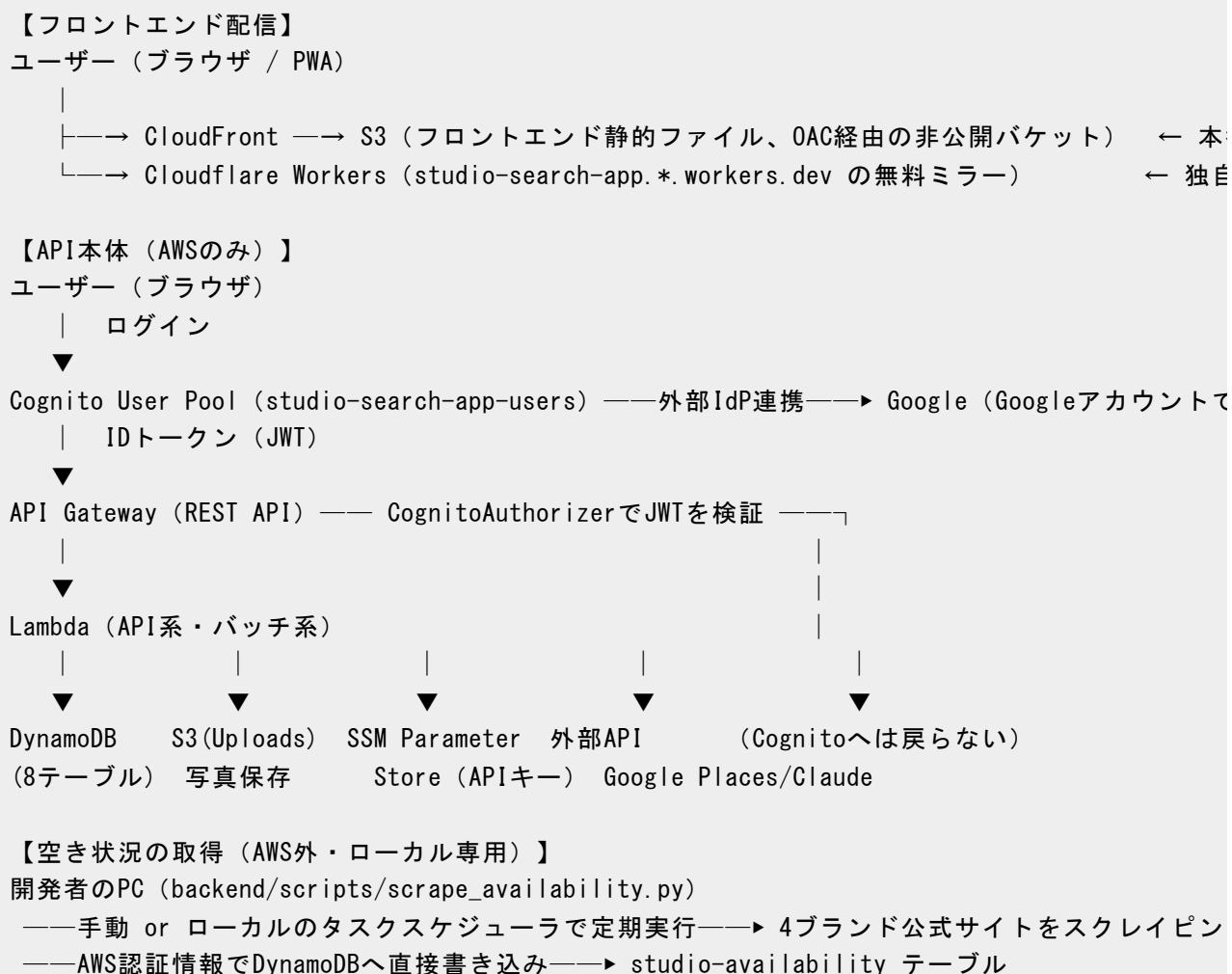


AWS構成・サービス連携ガイド（レンタルスタジオサーチアプリ）

このアプリで実際に使っているAWS（およびAWS外の関連）サービスを一覧化し、「何のために」「なぜそれを選んだか」「他のどのサービスと連携しているか」をサービス単位でまとめたもの。Lambda関数単位の詳細は[詳細設計.html](#)、ファイル単位の対応は[ファイル構成・依存関係.html](#)を参照。本資料は、対象を4ブランド（BUZZ / studio worcle / サウンドスタジオNOAH / スタジオミッション）の実店舗に絞り、AIによるスコアリング機能を廃止した後の構成を反映している（旧構成との差分は企画・設計アーカイブ.html参照）。

1. 全体構成図



(Lambdaとしてはデプロイしない。クラウド上の定期実行なし)

【定期実行 (AWS側)】

EventBridge (毎日夜間) → sendAnalyticsDigestBatch (Lambda、利用状況の集計メール送信)

【コスト監視】

AWS Budgets → メール通知 (月\$10の80%/100%到達時)

2. サービス一覧 (早見表)

サービス	役割	主な連携先
Amazon Cognito	Googleアカウントでのログイン認証 (釣行AIアプリとは別のUser Pool)	Google (外部IdP)、API Gateway (JWT検証)
Amazon API Gateway	フロントからのHTTPリクエストの受付・認証・Lambdaへのルーティング	Cognito、Lambda (API系Lambda群)
AWS Lambda	実際の処理を行うプログラム本体 (API系・バッチ系の2パッケージ)	DynamoDB、S3、SSM、外部API、他のLambda (非同期起動)
Amazon DynamoDB	アプリの全データ (スタジオ・空き状況・イベント・投稿・チャット等) の保存先	Lambda、backend/scripts/scrape_availability.py (ローカルから直接書き込み)
Amazon S3	①フロントエンドの静的ファイル配信 ②ユーザーがアップロードする写真の保存	CloudFront (OAC経由、①)、Lambda・ブラウザ直PUT (②)
Amazon CloudFront	フロントエンドをCDN経由で高速配信 (本番の正系)	S3 (Origin Access Control経由のオリジン)
Cloudflare Workers (AWS外)	フロントエンドの無料ミラー配信 (独自ドメイン未取得のため)	フロントのビルド成果物を直接デプロイ (AWSとは独立、Worker名studio-search-app)
AWS Systems Manager	外部APIキー (Claude・Google Places・Google	Lambda (実行時に取得)。パスは/studio-search/*で釣行AIアプリ (/fishing-ai/*) とは

Parameter Store (SSM)	OAuth) の安全な保管	分離
Amazon EventBridge	毎日決まった時刻に集計バッチLambdaを自動起動するスケジューラ	sendAnalyticsDigestBatch (Lambda)
AWS Budgets	AWS利用料が閾値を超えたらメール通知	Eメール
AWS IAM	各Lambdaに必要最小限の権限だけを付与	全AWSサービスへのアクセス制御
GitHub Actions (AWS外)	mainブランチへのpushをトリガーに自動デプロイ	AWS (SAM CLI経由)、Cloudflare、S3、CloudFront

ローカル専用スクリプトはAWSサービスとしては扱わない:

backend/scripts/scrape_availability.pyは4ブランドの公式サイトから空き状況・広さ・部屋写真・平面図・設備をスクレイピングしてstudio-availabilityテーブルへ直接書き込むが、Lambdaとしてはデプロイせず、開発者のローカル端末のAWS認証情報を使って 手動またはローカルのタスクスケジューラで実行する。クラウド上に常駐する処理ではないため、上表の「AWS サービス」には含めていない。

3. 各サービスの詳細

■ Amazon Cognito (User Pool + Google連携)

ログイン機能そのもの。「Googleアカウントでログイン」ボタンを押すとCognitoのHosted UIに飛び、Cognitoが裏でGoogleと認可コードのやり取りをして、最終的にアプリ向けのIDトークン(JWT)を発行する。このJWTがAPI Gatewayでの認証に使われる。Hosted UIドメインは studio-search-app-<AWSアカウントID>.auth.ap-northeast-1.amazonaws.com。

なぜ釣行AIアプリと別のUser Poolか: Client ID・Hosted UIドメインは User Pool単位でしか発行できず、共有するとログイン画面のURLやリダイレクト設定がアプリ間で衝突する。Google Cloud Console側のOAuthクライアントも専用のものを別途発行している。

■ Amazon API Gateway (REST API)

フロントエンドが呼び出す全エンドポイント

(/studios、/studios/{studioId}/availability、/events、/recommend-studios、/chat、/posts等) の窓口。ログイン必須のエンドポイントには CognitoAuthorizer をアタッチしており、JWTが無効・未提示なら該当Lambdaを呼ばずに401を返す。

なぜAPI Gateway: Lambdaを直接インターネットに公開せず、認証・CORS・ルーティングを一括管理できる。統合タイムアウトが29秒固定という制約があり、これが AIチャット（同期呼び出し、Lambda側25秒に短縮）の設計判断に直接影響している（詳細設計.html参照）。

■ AWS Lambda（API系・バッチ系の2パッケージ）

実処理の本体。backend/lambda/api/（handlers.py）と backend/lambda/batch/（discover_studios.py・admin_trigger.py・send_analytics_digest.py）の2つのSAM CodeUriパッケージに分かれており、この2つの間でコードは共有できない（batch_common.pyのようなヘルパーが両パッケージに重複して存在する理由）。以前存在していたスコア生成バッチ（generate_studio_score.py／generateStudioScoreBatch）は、AIスコアリング機能の廃止にともない完全に削除した。

なぜLambda（サーバーレス）: 個人利用規模のトラフィックでは「常時起動のサーバー」は過剰投資になる。呼ばれた時だけ課金される従量課金と、インフラ管理不要な点を重視した。

Lambda関数名を固定しなかった理由: 当初FunctionNameを 釣行AIアプリと同じ命名で固定していたところ、Lambda関数名はリージョン内でアカウント共通の一意制約があるため、既に稼働している釣行AIアプリの同名関数と衝突し、初回デプロイが AWS::EarlyValidation::ResourceExistenceCheckエラーで失敗した。全Lambdaの FunctionName指定を削除し、CloudFormationに自動採番させる形に修正することで 解消した。

■ Amazon DynamoDB（8テーブル）

テーブル名	役割
studio-studios	4ブランドの店舗マスタ（座標・名前・公式サイトURL・部屋マスタ配列。部屋ごとの広さ・料金・予約リンク・写真・平面図・設備を含む）
studio-availability	日付ごとの空き状況（30分刻みのタイムスロット）。scrape_availability.pyがローカルから書き込む
studio-events	ユーザーが登録する「次のイベント」（イベント名・ステージ横幅／奥行き・出演人数）。新規テーブル

studio-favorites	お気に入り登録
studio-posts	レビュー投稿（本文・写真・★評価）
studio-chats	AI相談チャットの会話履歴
studio-usage	1日あたりのAI/API呼び出し回数（レート制限用、TTLで自動削除）
studio-users	ユーザーの表示名（ユーザー名）
studio-analytics-events	アプリ内の操作イベントログ（集計バッチが日次でダイジェストメールを送るための元データ、TTLで自動削除）

studio-recommendations テーブルは削除済み。 AIスコア・おすすめTOP3を保存していた テーブルだが、スコアリング機能の廃止にともない完全に削除した。かわりに studio-availability（空き状況）とstudio-events（次のイベント）を 新設している。

なぜDynamoDB: スキーマがシンプルでアクセスパターンも 単純（PK検索・PK+SK範囲検索・全件scan中心）なため、RDBのような複雑なクエリは不要。PAY_PER_REQUESTモードで使った分だけ課金され、サーバーレスLambdaとの相性も良い。

■ Amazon S3（2バケット）

①フロントエンド配信用バケット（studio-search-app-frontend-〈アカウントID〉）：Reactのビルド成果物（HTML/JS/CSS）を置き、CloudFrontがOrigin Access Control（OAC）経由で 参照する非公開バケット。②UploadsBucket（studio-search-app-uploads-〈アカウントID〉）：ユーザーが投稿・スタジオに設定する写真の保存先。フロントはLambda（presignUploadHandler）から発行される署名付きPOSTフォームを使ってブラウザから直接S3へアップロードし、Lambda自体は画像バイナリを経由しない（API Gatewayのペイロードサイズ制限回避のため）。

なぜOACで非公開バケットか: S3の静的website機能（公開バケット）ではなく、CloudFrontのOrigin Access Control経由でのみ読み取りを許可するバケットポリシーにしている。バケット自体への直接アクセスを防ぎつつ、CloudFront経由のアクセスだけを許可できる、より安全な構成のため（釣行AIアプリの既存構成を踏襲）。

なぜ署名付きアップロード: 画像バイナリをLambda経由にすると API Gatewayのペイロード上限に引っかかる。S3への直接アップロードにすることで この制約を回避しつつ、content-length-range条件でアップロード上限（8MB）をS3側に強制させている。

■ Amazon CloudFront + Cloudflare Workers（フロント配信が2系統ある理由）

本番の正系はCloudFront（S3をオリジンとするCDN、OAC経由）。index.htmlはno-cache、ハッシュ付きアセット（/assets/index-`<hash>`.js）は長期キャッシュという構成。加えて、独自ドメインを取得する前の段階で見た目の良いURLを用意するため、Cloudflare Workers（studio-search-app.*.workers.dev）にも同じビルド成果物を並行デプロイしている。バックエンドAPI（API Gateway）はAWS側1本のみで共有しており、フロントの配信経路だけが2系統ある形。

釣行AIアプリのWorkerと衝突しない理由: wrangler.jsoncのnameをstudio-search-app（釣行AIアプリはryu-chan-fish）と明示的に別名にしており、GitHub Secrets（CLOUDFLARE_API_TOKEN等）もリポジトリごとに別登録のため、どちらかのアプリのデプロイがもう片方のWorkerを誤って上書きすることは無い。

■ AWS Systems Manager Parameter Store（SSM）

Claude APIキー（/studio-search/anthropic-api-key）、Google Places APIキー（/studio-search/google-places-api-key）、Google OAuthクライアントID/シークレット（/studio-search/google-oauth-client-id・-secret）をSSMに保管。Lambdaの環境変数に直書きせず、実行時にssm:GetParameterで取得する。ミッションブランドの会員ログイン情報（MISSION_LOGIN_EMAIL／MISSION_LOGIN_PASSWORD）はSSMではなく、ローカル専用のbackend/.env（Gitには含めず.env.exampleのみコミット）に保存しており、scrape_availability.pyがローカル実行時にのみ読み込む。

なぜ釣行AIアプリと別パスか: AnthropicのAPIキー自体（クレジット残高）は釣行AIアプリと共有してよい前提だが、Lambdaごとの権限スコープ（IAMポリシーが参照するパラメータ名）を綺麗に保つため、登録パス自体は/fishing-ai/*とは分離している。

■ Amazon EventBridge

毎日決まった時刻にsendAnalyticsDigestBatchを自動起動し、アプリの利用状況を集計してダイジェストメールを送る。以前存在していた「毎日AM6:00 JSTにスコアを再計算するEventBridgeルール」は、スコアリング機能の廃止にともない削除した。

■ AWS Budgets

月\$10のAWS利用料予算を設定し、80%/100%到達時にメールで通知する

(`template.yaml`の`CostBudget`リソース、予算名 `studio-search-app-monthly`)。ただしこれはAWS側の課金のみを見ており、Google Cloud (Places/Maps) やAnthropic (Claude) の課金はAWSの外で発生するため対象外。それぞれGoogle Cloud ConsoleとAnthropic Console側で別途予算アラート・`spend limit`の 設定が必要 (README.md参照)。

■ AWS IAM (最小権限方針)

各Lambda関数には、その関数が実際に必要とするDynamoDBテーブル・S3バケット・SSMパラメータへのアクセス権限だけをSAMのPolicies (`DynamoDBCrudPolicy`・`DynamoDBReadPolicy`・`SSMParameterReadPolicy`等) で個別に付与している。例えば `GetStudiosFunction`はStudiosテーブルの読み取りしかできず、書き込みや他テーブルへのアクセス権限は持たない。

4. 主要フローのサービス連携トレース

① ログイン

ブラウザ「ログイン」クリック

- Cognito Hosted UI (`studio-search-app-<アカウントID>.auth.ap-northeast-1.amazoncognito.`)
- Google認証画面 (Cognitoが仲介)
- Cognitoに戻りIDトークン (JWT) 発行
- フロントがJWTを保持し、以後のAPIリクエストにAuthorizationヘッダーとして付与

② スタジオ一覧・空き状況の閲覧

フロント (TopPage) → API Gateway → `getStudiosHandler` (Lambda)

- `StudiosTable`から4ブランドのスタジオ一覧を取得 (未ログインでも可能)

フロント「空き状況を見る」→ API Gateway → `getStudioAvailabilityHandler` (Lambda)

- `AvailabilityTable`から該当studioId・日付のデータを取得
(データが無い日は `rooms: []` / `scrapedAt: null` を返し、エラーにはしない)

③ 空き状況の取得 (ローカル、AWS外)

開発者PC上で `python backend/scripts/scrape_availability.py` を実行

- BeautifulSoup (静的HTML) またはPlaywright (JS描画・会員ログインが必要なサイト) で4ブランドの公式サイトから空き状況・広さ・写真・平面図・設備を取得

→ ローカルのAWS認証情報を使い、DynamoDB (AvailabilityTable) へ直接put_item (API Gateway/Lambdaを経由しない)

④ イベント登録とおすすめ検索

フロント (EventsPage) → API Gateway (CognitoAuthorizerでJWT検証)
→ postEventsHandler (Lambda) → EventsTableへ保存
フロント「おすすめを探す」 (目的・日付・時間帯を指定)
→ API Gateway → getRecommendedStudiosHandler (Lambda)
→ EventsTableからイベント (ステージ実寸・出演人数) を取得
→ 目的 (振り入れ=2.5㎡/人、構成=1.8㎡/人) から必要な広さを算出
→ StudiosTableの部屋マスタから広さ条件を満たす部屋を抽出
→ AvailabilityTableで指定日時のslotsを確認し、空きのある部屋だけに絞り込み
→ 条件を満たす部屋一覧を返却 (スコアリングはしない単純な絞り込み)

⑤ AI相談チャット (画像添付あり)

フロント → API Gateway (CognitoAuthorizerでJWT検証) → postChatHandler
→ UsageTableでレート制限チェック (1日30件)
→ S3(Uploads)から画像バイトを取得
→ StudiosTableをscanして実データのコンテキストを構築
→ Claude APIへテキスト+画像+実データコンテキストを送信 (SSMからキー取得)
→ 応答をChatsTableに保存し、同期的にフロントへ返却

⑥ 写真アップロード (レビュー投稿・スタジオ写真)

フロント → presignUploadHandler (Lambda) が署名付きPOSTフォームを発行
→ ブラウザがS3(Uploads)へ直接POST (Lambda/API Gatewayを経由しない)
→ アップロード完了後、公開URLをPosts/StudiosテーブルにLambda経由で保存

5. なぜこの構成にしたか (設計方針まとめ)

- **サーバーレス徹底:** 個人利用規模のトラフィックに対して常時稼働のサーバーは過剰。使った分だけ課金されるLambda・DynamoDB・S3 (+CloudFront) で構成し、待機コストをゼロに近づけている
- **スクレイピングはクラウドに乗せない:** 4ブランドの公式サイトのが構造が大きく異なり、一部は会員ログインやJavaScript描画への対応が必要なため、Lambdaでの常時運用ではなく ローカル専用スクリプト+手動/タスクスケジューラ実行という割り切った構成にしている

- **実データとAIの役割の縮小:** スコアリング機能の廃止にともない、AI（Claude）の役割はAI相談チャットの応答生成のみに縮小した。座標・実店舗の実在性など「間違えられないデータ」は Google Places APIや公式サイトの実データのみを情報源とし、AIには生成させない方針は維持している
- **最小権限のIAM:** Lambdaごとに必要なリソースへのアクセス権限だけを絞って付与
- **コスト保護の多層化:** ①ユーザー単位の1日あたりレート制限（UsageTable）、②AWS Budgets（AWS利用料）、③Google Cloud・Anthropic Console側の予算アラート（AWS外の課金）、という3層でコスト暴走を防ぐ設計にしている
- **既存アプリとの独立性:** 釣行AIアプリと同じAWSアカウント・同じAnthropic/Cloudflare アカウントを使い回しつつも、CloudFormationスタック・Cognito User Pool・SSMパラメータパス・Cloudflare Worker名・GitHub Secretsは全て別物にし、互いの運用に影響が出ないようにしている